Contents

1 Implementation Plan: Free-Surface (z = z_max) Slice VTK/PVD Output

1.1 Overview

Add a default-ON ParaView output that writes only the **free-surface boundary** of the SAFS/TPV domain, carrying the volume `velocity` field (and `mpi_rank`) sliced onto a codim-1 surface mesh. The slice is built once with `mfem::ParSubMesh::CreateFromBoundary` on the free-surface boundary attribute(s) and refreshed each output step via a cached `mfem::ParTransferMap`. It writes on its own fixed-dt schedule, independent of the volume/bulk/fault collections and the `paraview_enabled` master gate.

See `EXPLORE_volume_output_path.md` (same folder) for the traced volume-output path and the MFEM SubMesh feasibility verification. **Read it before implementing.**

1.2 Constraints

- **Interface (do not change):** the existing `seas::ParaViewOutput<ParMesh>` class, the `paraview_write` lambda's existing fault/bulk branches, the `Q` state layout (component-major, `VX=6` block at `Q.GetData()+VX*ndof_total`, length `3*ndof_total`, `ndof_total = wave.GetScalarNDof()`), and the checkpoint schema (no PV state persisted).
- **MFEM SubMesh prerequisite:** parent L2 fields must use `BasisType::GaussLobatto` (`submesh_utils.cpp:117–118` ASSERT). We satisfy this — mirror the existing `pv_vel_*` space construction exactly: `L2_FECollection(order, /*dim*/3, BasisType::GaussLobatto)`, `ParFiniteElementSpace(&pmesh, fec, /*vdim*/3, Ordering::byNODES)`.
- **MPI collectivity:** `ParSubMesh::CreateFromBoundary`, `ParTransferMap::Transfer`, and `DataCollection::Save` are all MPI-collective — **every rank must construct the slice and call Save**, including ranks with zero local free-surface faces (empty local submesh is valid).
- **Convention:** follow the existing config/CLI/banner patterns in `spatial_friction.{hpp,cpp}` and `spatial_dyn_driver.cpp`. Header-only writer like `paraview_output.hpp` (no new library `.o`).
- **Headers:** `ParSubMesh/ParTransferMap` are already aggregated via `mfem.hpp` (`mesh/mesh_headers.hpp:29,41`); the driver and the new header get them for free, but the new header

should still `#include "mfem.hpp"` explicitly.

- **Default-ON scope:** the slice writes for every run unless `paraview_free_surface="off"`, regardless of `paraview_enabled`/volume/bulk. It must NOT regress benchmark-only jobs that expect zero *other* ParaView files — it only ever creates `ParaView_free_surface/`.
- **Numerical:** the slice is a pure interpolation/trace of an existing field — no new physics, no tolerance. Transferred surface DOF values must equal the parent volume field's trace on the adjacent volume element (machine-eps; GaussLobatto nodal coincidence).

1.3 Phase 1: `FreeSurfaceOutput` writer class (header-only)

Goal

A self-contained `seas::FreeSurfaceOutput` class exists that, given a parent `ParMesh` and free-surface boundary attributes, builds a boundary submesh + transfer map + ParaView collection and can refresh/write the sliced velocity and mpi_rank fields — with no driver or config wiring yet.

Files to Create

- `miniapps/seas/io/free_surface_output.hpp` — the `FreeSurfaceOutput` class.

Detailed Requirements

1. Namespace `mfem::seas`. Guard `MFEM_USE_MPI` (class targets `ParMesh`; mirror `paraview_output.hpp` MPI handling). `#include "mfem.hpp"`, `<string>`, `<memory>`, `<filesystem>`.
2. Output-mode enum:

```
enum class Mode { Vtu, Hdf5 };
```

3. Constructor:

```
FreeSurfaceOutput(const std::string &prefix,       // e.g. "<out>/ParaView_free_surface"
                  ParMesh           &parent,
                  const Array<int>  &free_surface_attrs, // non-empty, validated by caller
                  int               order,
                  int               rank,
                  real_t            fixed_dt,
                  const std::string &collection_name = "free_surface",
                  Mode              mode = Mode::Vtu);
```

   Body, in this exact order (member declaration order must match for safe destruction — see §Edge Cases):

   1. `submesh_ = std::make_unique<ParSubMesh>(ParSubMesh::CreateFromBoundary(parent, free_surface_attrs));` (collective). Record `submesh_->GetNE()` reduced across ranks into `global_ne_`.
   2. Parent velocity space (must match Q layout): `parent_vel_fec_ = std::make_unique<L2_FECollection>(order, 3, BasisType::GaussLobatto); parent_vel_fes_ = std::make_unique<ParFiniteElementSpace>(&parent, parent_vel_fec_.get(), 3, Ordering::byNODES); parent_vel_gf_ = std::make_unique<ParGridFunction>(parent_vel_fes_.get()); *parent_vel_gf_ = 0.0;`
   3. Parent rank space: `L2_FECollection(0, 3)` on `&parent`, scalar `ParFiniteElementSpace`,

`parent_rank_gf_` set to `static_cast<real_t>(rank)`.

4. Submesh spaces (same FEC kinds, on `submesh_.get()`): `sub_vel_fec_` = `L2_FECollection(order, 3, GaussLobatto)`, `sub_vel_fes_` = vdim=3 byNODES on submesh, `sub_vel_gf_`. `sub_rank_fec_` = `L2_FECollection(0, 3)`, scalar on submesh, `sub_rank_gf_`.
5. Transfer maps (built once, reused): `vel_map_ = std::make_unique<ParTransferMap>(ParSubMesh::CreateTransferMap(*parent_vel_gf_, *sub_vel_gf_)); rank_map_ = std::make_unique<ParTransferMap>(ParSubMesh::CreateTransferMap(*parent_rank_gf_, *sub_rank_gf_));`
6. Transfer the static rank field once now: `rank_map_->Transfer(*parent_rank_gf_, *sub_rank_gf_);`
7. Data collection on the **submesh**:

```
if (mode == Mode::Hdf5) {
#ifdef MFEM_USE_HDF5
  pv_dc_ = std::make_unique<ParaViewHDFDataCollection>(collection_name, submesh_.get());
#else
  MFEM_ABORT("FreeSurfaceOutput Mode::Hdf5 needs MFEM_USE_HDF5=YES");
#endif
} else {
  pv_dc_ = std::make_unique<ParaViewDataCollection>(collection_name, submesh_.get());
}
pv_dc_->SetPrefixPath(prefix);
pv_dc_->SetDataFormat(VTKFormat::BINARY);
pv_dc_->SetHighOrderOutput(true);
pv_dc_->SetLevelsOfDetail(order);
pv_dc_->RegisterField("velocity", sub_vel_gf_.get());
pv_dc_->RegisterField("mpi_rank", sub_rank_gf_.get());
```

   Store `pv_dc_` as `std::unique_ptr<ParaViewDataCollectionBase>` (the abstract base used by `paraview_output.hpp`) so both back ends share the `SetCycle/SetTime/Save` call site.

8. `fixed_dt_ = fixed_dt; last_write_time_ = -1e30;`

4. Schedule check (read-only, fixed-dt; mirror `ParaViewOutput::kOutputTimeTolerance=0.99`):

```
static constexpr real_t kTol = 0.99;
bool ShouldWrite(real_t time) const
{ return (time - last_write_time_) >= fixed_dt_ * kTol; }
```

5. Velocity refresh — decoupled from the field-index enum (driver passes the raw block ptr):

```
void UpdateVelocity(const real_t *vxvyvz_block, int ndof_per_component)
{
   MFEM_VERIFY(parent_vel_fes_->GetNDofs() == ndof_per_component,
               "FreeSurfaceOutput: parent velocity NDofs mismatch");
   std::memcpy(parent_vel_gf_->GetData(), vxvyvz_block,
               3 * static_cast<size_t>(ndof_per_component) * sizeof(real_t));
   vel_map_->Transfer(*parent_vel_gf_, *sub_vel_gf_);
}
```

   (Rationale: `byNODES` vdim=3 GF data layout == `[VX block][VY block][VZ block]`, identical to the `Q.GetData()+VX*ndof_total` slice the volume path already memcpys at `spatial_dyn_driver.cpp:2734`.)

6. Write:

```
void Save(int cycle, real_t time)
{
   MFEM_VERIFY(pv_dc_, "FreeSurfaceOutput::Save: null collection");
   pv_dc_->SetCycle(cycle);
   pv_dc_->SetTime(time);
   pv_dc_->Save();
   last_write_time_ = time;
}
```

7. Accessors for tests/banner: `long long GlobalNE() const { return global_ne_; }`, `int Dimension() const { return submesh_->Dimension(); }`, `const ParaViewDataCollection-Base* GetDataCollection() const { return pv_dc_.get(); }`, `ParSubMesh& SubMesh() { return *submesh_; }`.
8. Member declaration order (top→bottom; destroyed bottom→top so `pv_dc_` dies before the submesh it points at): `submesh_`; parent spaces/gf; sub spaces/gf; `vel_map_`, `rank_map_`; `pv_-dc_`; PODs (`fixed_dt_`, `last_write_time_`, `global_ne_`). All MFEM objects via `std::unique_-ptr`.

Interfaces

- `seas::FreeSurfaceOutput` with the ctor, `ShouldWrite`, `UpdateVelocity`, `Save`, and the accessors above.

Edge Cases to Handle

- **Empty `free_surface_attrs`:** the **caller** must guarantee non-empty (the driver resolves + guards in Phase 3). Add `MFEM_VERIFY(free_surface_attrs.Size() > 0, ...)` as a defensive contract at ctor entry.
- **Rank with zero local free-surface faces:** `submesh_` local part is empty; all collective calls still execute. No special-casing; do NOT `return` early on any rank.
- **`global_ne_ == 0` (attribute present nowhere):** ctor still succeeds but the slice is empty; expose via `GlobalNE()` so the driver can warn. (Driver Phase 3 decides whether to skip construction; the class itself stays robust.)
- **`ParSubMesh` movability:** if `std::make_unique<ParSubMesh>(ParSubMesh::CreateFromBoundary(...))` fails to compile (no public move ctor), fall back to a heap factory: store `submesh_` and assign via the move; see Risk Assessment.

Acceptance Criteria

- ☐ `free_surface_output.hpp` compiles standalone (included in a TU that links MFEM).
- ☐ In a unit test (Phase 4): on a $2\times2\times2$ hex box with the top face tagged, the constructed slice has `Dimension()==2` and `GlobalNE() > 0`.
- ☐ After `UpdateVelocity` with a parent velocity GF set to a known field `v(x)=(x,y,z)`, the sub velocity GF values equal the parent trace at coincident surface nodes to within `1e-12` (Gauss-Lobatto nodal coincidence).
- ☐ `Save(0,0.0)` writes `ParaView_free_surface/free_surface.pvd` + a cycle VTU (Vtu mode) or `free_surface.vtkhdf` (Hdf5 mode); files exist and are non-empty.

Dependencies

- Depends on: nothing (uses only MFEM).
- Required by: Phase 3.

1.4 Phase 2: Config fields + TOML parse + validation

Goal

`[output]` accepts `paraview_free_surface` and `paraview_free_surface_dt`, with defaults that make the slice ON by default; parsing/validation mirror the existing paraview keys.

Files to Modify

- `miniapps/seas/spatial/code/spatial_friction.hpp` — extend `OutputSpec` (L161-189).
- `miniapps/seas/spatial/code/spatial_friction.cpp` — parse (after L1199) + validate (alongside L1209-1231).

Detailed Requirements

1. In `OutputSpec` add (after `paraview_interseismic_dt`, keep struct grouping/comment style):

```
// Free-surface slice (default ON; independent of the paraview master gate).
std::string paraview_free_surface    = "vtu";   ///< "off" | "vtu" | "hdf5"
real_t      paraview_free_surface_dt = 0.05;    ///< seconds; fixed cadence
std::vector<int> paraview_free_surface_attrs;   ///< optional override; empty ⇒ use [boundary].natural_attrs
```

   (Default `"vtu"` ⇒ on, VTK/PVD format, per decision (4).)

2. In the `[output]` parse block (mirror L1180-1185 idioms — `toml_str`, `toml_time_seconds`, and the existing `toml_int_array` used for boundary):

```
cfg.output.paraview_free_surface    = toml_str(o, "paraview_free_surface", "vtu");
cfg.output.paraview_free_surface_dt = toml_time_seconds(o, "paraview_free_surface_dt", 0.05);
toml_int_array(o, "paraview_free_surface_attrs", cfg.output.paraview_free_surface_attrs);
```

3. Validation (mirror L1209-1231):
   - `paraview_free_surface` ∈ {`"off"`,`"vtu"`,`"hdf5"`} — add to the existing mode-name loop or a parallel `MFEM_VERIFY`.
   - `MFEM_VERIFY(cfg.output.paraview_free_surface_dt > 0.0, ...)`.
   - Each entry of `paraview_free_surface_attrs` (if any) must be `> 0` (reuse the boundary-attr positivity check pattern at L1504).

Edge Cases to Handle

- Config omits both keys ⇒ defaults `"vtu"` + `0.05 s` (ON). Existing TOMLs that never mention these keys therefore newly emit a free-surface slice — intended (decision 3).
- `paraview_free_surface="off"` ⇒ disabled (Phase 3 skips construction).

Acceptance Criteria

- ☐ A parse unit test (Phase 4) confirms: default config ⇒ `paraview_free_surface=="vtu"`, `paraview_free_surface_dt==0.05`; `paraview_free_surface="off"` round-trips; an out-of-set value aborts; `dt<=0` aborts.
- ☐ Existing `spatial_friction` parse tests still pass.

Dependencies

- Depends on: nothing.

- Required by: Phase 3.

1.5 Phase 3: Driver wiring (CLI, construction, per-step write, banner)

Goal

`spatial_dyn_driver` constructs a `FreeSurfaceOutput` (default-ON), refreshes + saves it on its own dt inside `paraview_write`, and reports it in the rank-0 banner.

Files to Modify

- `miniapps/seas/drivers/spatial_dyn_driver.cpp`.

Detailed Requirements

1. **Include** (top, with the other io includes near L73): `#include "../io/free_surface_output.hpp"`.
2. **CLI flags** (mirror L576-601 / apply at L685-711):
   - `--paraview-free-surface <off|vtu|hdf5>` ⇒ overrides `cfg.output.paraview_free_surface`.
   - `--paraview-free-surface-dt <seconds>` ⇒ overrides `cfg.output.paraview_free_surface_dt` when `> 0`. Use the existing `GetStringArg/GetRealArg` helpers and the same override-ordering style.
3. **Mode parse helper** (near `ParseVolumeMode` L180): add

```
seas::FreeSurfaceOutput::Mode ParseFreeSurfaceMode(const std::string &s, bool &enabled) {
  if (s == "off")  { enabled = false; return seas::FreeSurfaceOutput::Mode::Vtu; }
  enabled = true;
  if (s == "vtu")  return seas::FreeSurfaceOutput::Mode::Vtu;
  if (s == "hdf5") return seas::FreeSurfaceOutput::Mode::Hdf5;
  MFEM_ABORT("spatial_dyn_driver: paraview_free_surface: unknown value '" << s << "'");
}
```

4. **Resolve free-surface attributes** (after the `BoundaryConfig bc` setup ~L940-947):

```
Array<int> fs_attrs;
const std::vector<int> &fs_src =
    !cfg.output.paraview_free_surface_attrs.empty()
      ? cfg.output.paraview_free_surface_attrs
      : cfg.boundary.natural_attrs;          // free surface = zero-traction natural BC
for (int a : fs_src) { fs_attrs.Append(a); }
```

5. **Construct `fs_out`** (a `std::unique_ptr<seas::FreeSurfaceOutput>`), placed AFTER `Q`, `wave`, `ndof_total` (L2511) are available and near the pv_out/pv_bulk_out block (L2231-2432). Logic:

```
bool fs_enabled = false;
auto fs_mode = ParseFreeSurfaceMode(cfg.output.paraview_free_surface, fs_enabled);
std::unique_ptr<seas::FreeSurfaceOutput> fs_out;
if (fs_enabled) {
  if (fs_attrs.Size() == 0) {
    if (rank == 0)
      std::cerr << "[free-surface] WARNING: paraview_free_surface on but no "
                   "free-surface attributes ([boundary].natural_attrs / "
                   "[output].paraview_free_surface_attrs both empty); slice disabled.\n";
  } else {
    const std::string fs_dir = cfg.output.output_dir + "/ParaView_free_surface";
```

```
    if (rank == 0) std::filesystem::create_directories(fs_dir);
#ifdef MFEM_USE_MPI
    MPI_Barrier(comm);
#endif
    fs_out = std::make_unique<seas::FreeSurfaceOutput>(
              fs_dir, pmesh, fs_attrs, cfg.mesh.order, rank,
              cfg.output.paraview_free_surface_dt, "free_surface", fs_mode);
    if (rank == 0 && fs_out->GlobalNE() == 0)
      std::cerr << "[free-surface] WARNING: 0 surface elements matched attrs "
                   "{...}; check natural_attrs.\n";
  }
}
```

Collectivity: `make_unique<FreeSurfaceOutput>` is collective — it must be reached by ALL ranks (guard only on `fs_enabled` + non-empty `fs_attrs`, both rank-invariant). Do not gate construction on per-rank conditions.

6. **Per-step write** — modify the `paraview_write` lambda (L2670-2766):
   - Capture `fs_out` (it is in scope via `[&]`).
   - Compute `const bool fs_wants = fs_out && fs_out->ShouldWrite(time);` alongside `fault_wants/bulk_wants`.
   - Change the early return (L2676) to `if (!fault_wants && !bulk_wants && !fs_wants) return;`.
   - Add a free-surface block (independent of fault/bulk; place it before or after the existing branches, but it must run whenever `fs_wants` even if fault/bulk do not):

     ```
     if (fs_wants) {
       fs_out->UpdateVelocity(Q.GetData() + VX * ndof_total, ndof_total);
       fs_out->Save(step_num, time);
     }
     ```

     `VX` (=6) and `ndof_total` (L2511) are in scope. This reuses the same Q velocity block the volume path memcpys at L2732-2737.
7. **Initial frame:** the pre-loop `paraview_write(0, cfg.time.t_initial, 0.0)` (L2770) now also emits the t0 free-surface frame because `ShouldWrite` is true at `last_write_time_ = -1e30`. Verify no double-write on the first loop step (the `0.99` tolerance + `last_write_time_` update prevents it — same mechanism as the volume writer).
8. **Banner** (rank-0, L2435-2461): add

   ```
   std::cout << "  FreeSurf: " << cfg.output.paraview_free_surface
             << " (dt=" << cfg.output.paraview_free_surface_dt << " s";
   if (fs_out) std::cout << ", " << fs_out->GlobalNE() << " surf elems";
   std::cout << ")\n";
   ```

   Print this even when `any_pv_requested` is false (the slice is independent of the master gate) — i.e., emit a minimal banner line whenever `fs_out` exists.

Edge Cases to Handle

- **Volume + bulk both off (SAFS default):** `fs_out` still constructed and writes — this is the headline default-ON behavior. `paraview_write` must NOT be short-circuited by the volume/bulk-off paths.
- **Restart:** no checkpoint change; `last_write_time_` resets to `-1e30`, so the slice writes its first

post-restart frame immediately (matches volume/fault behavior). Output goes to the restart run's distinct `--output-dir` (per project memory note).

- **`paraview_free_surface="off"`:** `fs_out` is null; `paraview_write` fs branch and banner line are skipped; zero `ParaView_free_surface/` files created.
- **2D meshes:** out of scope (driver is 3D); no special handling.

Acceptance Criteria

- ☐ Build `seas_spatial_dyn_driver` succeeds (`conda activate mfem-dev`; from worktree use `make MFEM_DIR=../.. MFEM_BUILD_DIR=$MAIN MFEM_INC_DIR=$MAIN MFEM_LIB_DIR=$MAIN ...` per memory note `worktree-build-mfem-dir-override`).
- ☐ A short smoke run (or the existing smoke harness) with a TPV/SAFS config (no paraview keys set) produces `ParaView_free_surface/free_surface.pvd` + $\geq$1 VTU, and the volume/bulk dirs remain absent (proving independence from the master gate).
- ☐ `paraview_free_surface="off"` $\Rightarrow$ no `ParaView_free_surface/` directory.
- ☐ No change to fault/volume/bulk outputs vs. the pre-change build for an identical config (diff the fault `.vtkhdf` / station traces — byte-identical or within machine-eps).

Dependencies

- Depends on: Phase 1, Phase 2.
- Required by: Phase 4 (driver-level acceptance).

1.6 Phase 4: Tests

Goal

Unit coverage for the writer class and the config parse; a driver-level check folded into the existing smoke verification.

Files to Create

- `miniapps/seas/tests/unit/test_free_surface_slice.cpp` — class-level test.
- (Parse coverage) extend an existing `spatial_friction` parse test, or add `miniapps/seas/tests/unit/test_free_surface_config_parse.cpp`.

Files to Modify

- `miniapps/seas/Makefile` — add `seas_test_free_surface_slice` target + `test-free-surface-slice` rule (mirror the existing `TEST_*` block pattern, e.g. L436-465 / L1961-2013 / L4454-4499). Note `makefile-no-header-deps-stale-o`: the test depends on `free_surface_output.hpp` (header-only) — list the header so a force-rebuild is obvious; no new lib `.o`.
- `miniapps/seas/safs/project_7.0_alternative/spatial/code/scripts/verify_spatial_dyn_smoke_safs.py` — add an assertion that `ParaView_free_surface/free_surface.pvd` exists with $\geq$1 dataset and the `velocity/mpi_rank` arrays are present (only when the smoke config leaves the slice default-ON).

Detailed Requirements

1. **`test_free_surface_slice.cpp`** (MPI-aware; pass under `mpirun -np 1` and `-np 2`):

- Build a small structured hex mesh (`Mesh::MakeCartesian3D(2,2,2, HEXAHEDRON)`), tag the `z==z_max` boundary faces with a distinct attribute (set `bdr_attribute` by looping boundary elements and testing face-center z), `SetAttributes()`, `ParMesh pmesh(MPI_COMM_WORLD, mesh)`.
- Construct `FreeSurfaceOutput fs(tmpdir, pmesh, {topattr}, order=2, rank, dt=0.05, "fs", Vtu)`.
- Assert `fs.Dimension()==2` and `fs.GlobalNE() == expected` (=4 top quads for a $2\times2\times2$ box, reduced across ranks).
- Create a parent velocity `ParGridFunction` on the SAME space the class builds (L2-GLL order 2, vdim 3, byNODES) set to `v(x)=(x0,x1,x2)` via a `VectorFunctionCoefficient`; call `fs.UpdateVelocity(parent_gf.GetData(), parent_fes.GetNDofs())`.
- Pull the sub velocity GF (`fs` exposes it via an added test accessor or via the registered field on `GetDataCollection()`); assert at every sub node the value equals the analytic `v` at that node's physical coordinate to `< 1e-12`.
- `fs.Save(0, 0.0)`; assert the `.pvd` and a `.vtu/.pvtu` exist and are non-empty; clean up the tmpdir.
- **Schedule:** assert `ShouldWrite(0.0)` true initially; after `Save(0,0.0)`, `ShouldWrite(0.01)` false and `ShouldWrite(0.05)` true ($\times0.99$ tol boundary).

2. **Parse test**: load an in-memory/temp TOML with no free-surface keys $\Rightarrow$ defaults (`"vtu"`, `0.05`); one with `paraview_free_surface="off"`; one with an illegal value $\Rightarrow$ expect abort (use the project's existing `EXPECT_ABORT`/death-test idiom if present, else a guarded `try` around the validate call).

Acceptance Criteria

- ☐ `make seas_test_free_surface_slice && ./seas_test_free_surface_slice` passes serially.
- ☐ Same test passes under `mpirun -np 2` (exercises empty-local-submesh ranks + collective transfer/Save).
- ☐ Parse test passes; existing unit suite unaffected.

Dependencies

- Depends on: Phase 1, Phase 2 (and Phase 3 for the smoke-script assertion).

1.7 Testing Strategy

- **Phase 1/4:** analytic-trace equivalence — a linear field's surface trace is exact at GaussLobatto nodes, so the parent↔sub transfer is verifiable to machine-eps with no reference solution needed. Geometry checks (`Dimension`, `GlobalNE`) catch attribute mis-selection.
- **Phase 2:** table-driven parse/validate tests mirroring existing `[output]` coverage.
- **Phase 3:** smoke run for file presence + independence from the master gate; regression diff of fault/volume/bulk artifacts to prove no behavioral change to existing outputs.
- **MPI:** `-np 2` exercises shared faces on the free surface and empty-local-submesh ranks, the two parallel risks.
- Per project policy (`feedback_no_local_mesh_runs`): verify via compile + unit tests locally; full SAFS production runs go to Frontera.

1.8 Risk Assessment

- **`ParSubMesh` move into `unique_ptr`:** `CreateFromBoundary` returns by value; storing via `std::make_unique<ParSubMesh>(ParSubMesh::CreateFromBoundary(...))` relies on a public move ctor. *Detect:* compile error in Phase 1. *Mitigation:* if it fails, hold `std::optional<ParSubMesh>`/placement, or default-construct the spaces lazily after a member `ParSubMesh submesh_;` assigned by move — keep the DataCollection pointing at a stable address (do NOT reseat after the collection is built).
- **DataCollection lifetime vs. submesh:** `pv_dc_` stores `Mesh*` to `submesh_`; member order (submesh first, pv_dc_ last) ensures correct destruction. *Detect:* ASAN/use-after-free at teardown. Covered by Phase 4 running to completion.
- **L2 basis mismatch:** if any future parent velocity space switches off GaussLobatto, the transfer ASSERTs (`submesh_utils.cpp:117`). *Mitigation:* the class hard-codes GaussLobatto for its own parent source GF, so it is self-consistent regardless of the volume path.
- **`natural_attrs` includes non-free-surface boundaries (e.g. a bottom zero-traction face):** the slice would then include those faces too. *Detect:* `GlobalNE()` larger than expected / visual check. *Mitigation:* the `[output].paraview_free_surface_attrs` override lets the user pin the exact attribute(s); document this in the config comment.
- **Default-ON surprises benchmark jobs:** any existing config now emits a small `ParaView_free_surface/` tree. *Detect:* smoke/regression. *Mitigation:* documented; opt-out via `paraview_free_surface="off"`. Disk cost is a 2D surface (negligible vs. volume).
- **Per-step transfer cost:** one `memcpy` (3·ndof) + one `ParTransferMap::Transfer` per slice write. Bounded by the slice dt (0.05 s default), far coarser than the fault dt (0.001 s), so cost is small. The transfer map and submesh are built once. “‘